Migration Checklist

Comprehensive checklists for planning and executing VM migrations with Hyper2KVM.

Quick Links

Pre-Migration Checklist

Planning Phase (1-2 Weeks Before)

1. Inventory and Assessment

- [ ] **Complete VM inventory**
  - [ ] List all VMs to migrate
  - [ ] Document VM specifications (CPU, RAM, disk)
  - [ ] Identify OS versions
  - [ ] Map dependencies between VMs
  - [ ] Prioritize migration order
- [ ] **Inspect source VMs**

```
# For each VM
./scripts/vmdk_inspect.py /path/to/vm.vmdk > inspection-report.txt
```

- [ ] Review inspection reports
- [ ] Identify potential issues
- [ ] Note special requirements (clones, multi-disk, etc.)

- [ ] **Capacity planning**
  - [ ] Calculate total storage required
  - [ ] Verify target infrastructure capacity
  - [ ] Plan network bandwidth requirements
  - [ ] Estimate migration time windows

2. Environment Preparation

- [ ] **Install Hyper2KVM**

```
pip install "hyper2kvm[full]"
h2kvmctl --version
```

- [ ] **Verify dependencies**

```
qemu-img --version
python3 --version
df -h  # Check disk space
free -h  # Check memory
```

- [ ] **Test migration environment**

- [ ] Test VM with similar config
- [ ] Verify network connectivity
- [ ] Test conversion speed
- [ ] Validate boot process

- [ ] **Prepare target infrastructure**
  - [ ] Configure KVM host(s)
  - [ ] Setup storage (SAN, NFS, local)
  - [ ] Configure networking
  - [ ] Install libvirt/QEMU

3. Access and Permissions

- [ ] **Source access**
  - [ ] ESXi/vCenter credentials
  - [ ] SSH keys configured
  - [ ] VMDK file access
  - [ ] Network connectivity verified

- [ ] **Target access**
  - [ ] KVM host access
  - [ ] Storage write permissions
  - [ ]

- [ ] libvirt permissions
- [ ] Network configuration rights

4. Documentation

- [ ] **Create migration plan**
  - [ ] Migration schedule
  - [ ] VM migration order
  - [ ] Downtime windows
  - [ ] Rollback plan
  - [ ] Contact list (escalation)
- [ ] **Prepare configurations**
  - [ ] YAML configs for each VM
  - [ ] Batch manifests if applicable
  - [ ] Test configurations validated

5. Backup and Safety

- [ ] **Backup source VMs**
  - [ ] Create snapshots on VMware
  - [ ] Export VMDKs to safe location
  - [ ] Document backup locations
  - [ ] Test restore procedure

- [ ] **Prepare rollback plan**
  - [ ] Document rollback steps
  - [ ] Identify rollback decision points
  - [ ] Prepare rollback scripts
  - [ ] Test rollback procedure

Migration Day Checklist

Pre-Migration (1 Hour Before)

- [ ] **Final preparations**
  - [ ] Verify all prerequisites met
  - [ ] Confirm team availability
  - [ ] Test communication channels
  - [ ] Review migration plan
- [ ] **System checks**

```
# Verify disk space
df -h /output/directory

# Check system load
uptime

# Verify network
ping -c 4 esxi-host.example.com
```

- [ ]

Backup verification

- [ ] Confirm recent backups exist
- [ ] Verify backup integrity
- [ ] Document backup timestamps

During Migration

For Each VM:

- [ ] **Pre-migration**
  - [ ] Record VM current state
  - [ ] Note running services
  - [ ] Document IP address/network config
  - [ ] Take final snapshot
- [ ] **Shutdown source VM**
  - [ ] Graceful shutdown
  - [ ] Verify VM is stopped
  - [ ] Record shutdown time
- [ ] **Execute migration**

```
# Start migration
h2kvmctl --config vm-migration.yaml
```

```
# Monitor progress
tail -f /var/log/hyper2kvm/migration.log
```

- [ ] **Migration progress tracking**
  - [ ] Record start time
  - [ ] Monitor disk I/O
  - [ ] Monitor network transfer
  - [ ] Watch for errors
- [ ] **Post-conversion validation**

```
# Verify output file
qemu-img info /output/vm.qcow2
qemu-img check /output/vm.qcow2

# Check size
ls -lh /output/vm.qcow2
```

Quality Checks:

- [ ] **File integrity**
  - [ ] QCOW2 file created
  - [ ] File size reasonable
  - [ ] No corruption detected
  - [ ] Checksum calculated
- [ ] **Boot test**

```
# Test boot (if libvirt_test enabled)
virsh list --all
virsh start test-vm
virsh console test-vm
```

- [ ] **Initial validation**
  - [ ] VM boots successfully
  - [ ] OS loads without errors
  - [ ] No kernel panics
  - [ ] Basic functionality works

Post-Migration Checklist

Immediate Post-Migration (Within 1 Hour)

- [ ] **Import to production**

```
# Create libvirt domain
virsh define vm.xml

# Start VM
virsh start vm-name

# Verify status
virsh list
```

- [ ] **Boot validation**
  - [ ] VM boots successfully
  - [ ] No errors in boot logs

- [ ] OS fully loads
- [ ] Login successful

- [ ] **System validation**
  - [ ] All filesystems mounted
  - [ ] Network configured correctly
  - [ ] All services started
  - [ ] No error messages

Linux VMs:

- [ ] **Check boot logs**

```
# Console access
virsh console vm-name

# Check logs
journalctl -b
dmesg | grep -i error
```

- [ ] **Verify fstab**

```
cat /etc/fstab
mount | grep -v tmpfs
df -h
```

- [ ] **Check networking**

```
ip addr show
ip route show
ping -c 4 8.8.8.8
```

- [ ] **Verify services**

```
systemctl status
systemctl list-units --failed
```

Windows VMs:

- [ ] **Check Device Manager**
  - [ ] No unknown devices
  - [ ] VirtIO drivers loaded
  - [ ] Network adapter present
  - [ ] Disk controller correct
- [ ] **Verify drivers**
  - [ ] VirtIO SCSI driver
  - [ ] VirtIO network driver
  - [ ] VirtIO balloon driver
  - [ ] VirtIO serial driver
- [ ] **Test network**

```
ipconfig /all
ping 8.8.8.8
Test-Connection google.com
```

Detailed Validation (Within 4 Hours)

- [ ] **Application testing**
  - [ ] All applications start
  - [ ] Database connectivity works
  - [ ] Web services respond
  - [ ] Critical functions tested
- [ ] **Performance validation**
  - [ ] CPU usage normal
  - [ ] Memory usage acceptable
  - [ ] Disk I/O performance good
  - [ ] Network throughput adequate
- [ ] **Security validation**
  - [ ] Firewall rules applied
  - [ ] SELinux/AppArmor status
  - [ ] SSH access working
  - [ ] User permissions correct
- [ ]

Monitoring setup

- [ ] VM added to monitoring
- [ ] Alerts configured
- [ ] Metrics collecting
- [ ] Logs aggregating

Extended Validation (Within 24 Hours)

- [ ] **Full functionality test**
  - [ ] Run test suite if available
  - [ ] Verify all integrations
  - [ ] Test backup procedures
  - [ ] Validate HA/DR setup
- [ ] **Performance benchmarking**

```
# Disk performance
fio --name=randwrite --ioengine=libaio --iodepth=16 \
    --rw=randwrite --bs=4k --direct=1 --size=1G \
    --numjobs=4 --runtime=60

# Network performance
iperf3 -c target-host
```

- [ ] **User acceptance testing**
  - [ ] Users can access VM
  - [ ] Applications work as expected

- [ ] Performance acceptable
- [ ] No complaints from users

Rollback Checklist

Decision Point

- [ ] **Evaluate if rollback needed**
  - [ ] Migration failed?
  - [ ] VM won't boot?
  - [ ] Critical functionality broken?
  - [ ] Performance unacceptable?
  - [ ] User acceptance issues?

Rollback Execution

- [ ] **Stop migrated VM**

```
virsh shutdown vm-name
# If doesn't respond
virsh destroy vm-name
```

- [ ] **Restore source VM**

```
# On VMware
# 1. Remove snapshot if migration successful
```

```
# 2. Or restore from snapshot if VM was modified
# 3. Power on source VM
```

- [ ] **Verify source VM**
  - [ ] VM boots successfully
  - [ ] All services running
  - [ ] Network connectivity restored
  - [ ] Applications functional
- [ ] **Redirect traffic**
  - [ ] Update DNS if changed
  - [ ] Restore load balancer config
  - [ ] Update monitoring
  - [ ] Notify users
- [ ] **Document rollback**
  - [ ] Reason for rollback
  - [ ] Time of rollback
  - [ ] Issues encountered
  - [ ] Lessons learned

Post-Rollback

- [ ]

Analyze failure

- [ ] Review logs
- [ ] Identify root cause
- [ ] Document issues
- [ ] Plan remediation

- [ ] **Plan retry**
  - [ ] Fix identified issues
  - [ ] Update migration config
  - [ ] Test in lab environment
  - [ ] Schedule retry

Production Cutover Checklist

Pre-Cutover (1 Day Before)

- [ ] **Final validation**
  - [ ] All VMs migrated successfully
  - [ ] All testing complete
  - [ ] Performance acceptable
  - [ ] No critical issues

- [ ] **Cutover planning**

- [ ] Cutover schedule finalized
- [ ] Team notified
- [ ] Users notified
- [ ] Rollback plan confirmed

- [ ] **Communication**
  - [ ] Send cutover notification
  - [ ] Confirm team availability
  - [ ] Schedule status meetings
  - [ ] Prepare status updates

During Cutover

- [ ] **Network cutover**
  - [ ] Update DNS records
  - [ ] Modify load balancer config
  - [ ] Update firewall rules
  - [ ] Verify routing

- [ ] **Application cutover**
  - [ ] Update connection strings
  - [ ] Modify configuration files
  - [ ]

- [ ] Restart dependent services
- [ ] Clear caches

- [ ] **Monitoring cutover**
  - [ ] Update monitoring configs
  - [ ] Verify metrics collection
  - [ ] Test alerts
  - [ ] Update dashboards

- [ ] **Validation**
  - [ ] DNS resolves correctly
  - [ ] Applications accessible
  - [ ] Services responding
  - [ ] No errors in logs

Post-Cutover

- [ ] **Production validation**
  - [ ] All systems operational
  - [ ] Users can access services
  - [ ] Performance acceptable
  - [ ] No critical errors

- [ ]

Cleanup

- [ ] Document final state
- [ ] Archive migration files
- [ ] Remove temporary configs
- [ ] Update documentation

- [ ] **Decommission source**
  - [ ] Keep source VMs for rollback window
  - [ ] Schedule decommissioning
  - [ ] Archive source data
  - [ ] Release resources

Special Scenarios Checklists

Database Server Migration

Additional checks:

- [ ] **Pre-migration**
  - [ ] Stop database gracefully
  - [ ] Verify clean shutdown
  - [ ] No active connections
  - [ ] Backup database files

- [ ] **Post-migration**
  - [ ] Database starts successfully
  - [ ] Run integrity checks
  - [ ] Verify replication (if applicable)
  - [ ] Test query performance
  - [ ] Validate backups work

Windows Domain Controller

Additional checks:

- [ ] **Pre-migration**
  - [ ] Verify replication status
  - [ ] Check FSMO roles
  - [ ] Document DNS settings
  - [ ] Note time sync source
- [ ] **Post-migration**
  - [ ] AD services running
  - [ ] Replication working
  - [ ] DNS resolving
  - [ ] Time sync functional
  - [ ] Authentication working

Multi-Disk VMs

Additional checks:

- [ ] **Pre-migration**
  - [ ] Document all disk mappings
  - [ ] Note mount points
  - [ ] Check for disk dependencies
- [ ] **Post-migration**
  - [ ] All disks attached
  - [ ] All filesystems mounted
  - [ ] Correct mount points
  - [ ] Data accessible

Checklist Templates

Simple VM Migration Template

```
VM Name: _________________
OS: _________________
Size: _________________
Priority: _________________

Pre-Migration:
[ ] Backup created: _________________
[ ] Inspection done: _________________
[ ] Config prepared: _________________

Migration:
[ ] Started: _________________
```

```
[ ] Completed: _______________
[ ] Output verified: _______________

Post-Migration:
[ ] Boot test: _______________
[ ] Network test: _______________
[ ] App test: _______________
[ ] Sign-off: _______________
```

Batch Migration Template

```
Batch Name: _______________
Number of VMs: _______________
Schedule: _______________

Pre-Migration:
[ ] All VMs backed up
[ ] All configs prepared
[ ] Capacity verified
[ ] Team notified

Migration:
[ ] Batch started: _______________
[ ] VMs completed: _____ / _____
[ ] Errors: _______________

Post-Migration:
[ ] All VMs booted
[ ] All VMs validated
[ ] All apps tested
[ ] Batch sign-off: _______________
```

Downloadable Checklists

Print-Friendly Versions

Create simple text files from templates above:

```
# Generate migration checklist for VM
cat > migration-checklist-${VM_NAME}.txt << 'EOF'
MIGRATION CHECKLIST - ${VM_NAME}
================================
```

```
PRE-MIGRATION
[ ] Backup created
[ ] Inspection complete
[ ] Config validated
[ ] Team notified

MIGRATION
[ ] Start time: _______
[ ] End time: _______
[ ] Status: _______

POST-MIGRATION
[ ] Boot validated
[ ] Network validated
[ ] Applications tested
[ ] Sign-off: _______
EOF
```

Automated Validation Scripts

Quick Validation Script

```
#!/bin/bash
# quick-validate.sh - Post-migration validation

VM_NAME=$1

echo "=== Quick Validation for $VM_NAME ==="

# Check VM status
echo -n "VM Status: "
virsh domstate $VM_NAME

# Check if VM is running
if [ "$(virsh domstate $VM_NAME)" != "running" ]; then
    echo "ERROR: VM is not running"
    exit 1
fi

# Check console output for errors
echo "Checking for errors..."
virsh console $VM_NAME --force &
CONSOLE_PID=$!
```

```
sleep 5
kill $CONSOLE_PID 2>/dev/null

# Check domain XML
echo "Checking configuration..."
virsh dumpxml $VM_NAME | grep -q "qcow2" && echo "✓ Using qcow2"

# Check network
echo "Checking network..."
virsh domifaddr $VM_NAME

echo "=== Validation Complete ==="
```

Success Criteria

Individual VM Migration Success

A migration is successful when:

- VM boots without errors
- All filesystems mounted
- Network configured correctly
- All services started
- Applications functional
- Performance acceptable
- No data loss
- User acceptance confirmed

Batch Migration Success

A batch migration is successful when:

- All VMs migrated
- All VMs validated
- No critical failures
- Performance acceptable
- Within schedule
- Documentation complete

- Team sign-off obtained

Last Updated: February 2026 **Documentation Version**: 2.1.0